Smart Light: A Distributed On-device Voice-controlled LED Prototype

Team Members: Qingtong You, Qiying Chen, Junzhang Chen, Siwei Jiang

Overview of the Project

Voice recognition, which provides a convenient, hands-free interaction method in many home scenarios like cooking, cleaning or carrying stuff, has become increasingly integrated into smart home systems. However, collecting and processing users' voice data in the cloud can raise privacy concerns, which could expose personal identity or living habits. At the same time, dialectal variations also make it difficult for a single pre-trained model to perform well for all users, resulting in degraded accuracy and inconsistent user experiences.

To address these issues, this project aims to implement a small-scale prototype of distributed on-device training voice recognition system using Arduino TinyML Kit. A federated learning framework is employed here, where all devices start with the same light-weight model with random weights. In every federated round, each device starts from the same global model and then performs 3 epochs of on-device training using its own private data. Afterwards, these devices share only the updated model weights, rather than any raw audio data, to an aggregation node. This node applies federated averaging to combine the received weights into a new global model and subsequently distributes the model back to all participating devices. This cycle is repeated for 10 federated rounds, allowing devices to collaboratively improve a shared model while ensuring that users' voice data never leaves their own hardware.

Plan, Tasks, and Responsibilities

I. Task 1: Data Collection (Week 48, Contributor: Siwei Jiang & Junzhang Chen)

Voice data from two users, one male and one female, were collected using the built-in microphone on the MCUs, in order to reflect realistic on-device usage conditions. The data collection was process assisted by Edge Impulse for quality inspection convenience. Simple pre-processing operations were performed here as well, including audio segmentation, trimming all samples to same length of 500 ms at 16,000 Hz (i.e., 8000 points), and temporal alignment to the onset of the speech signal.

This step produced labeled and relatively well-structured dataset for subsequent on-device training and federated learning implementation. In total, 260 valid samples were collected from User 1 and 200 valid samples from User 2. The int16 PCM audio data within these exported .wav files was extracted and stored as C arrays, serving as an intermediate representation for subsequent feature extraction and use on the MCUs.

II. Task 2: ML Algorithm and Implementation on IoT Sensor (Week 49-50, Contributor: Qingtong You & Qiying Chen)

In this task, the on-device classical neural network pipeline was implemented on the Arduino-based MCUs, together with the BLE-based federated framework. All local model training was performed entirely on the MCUs, while the PC participated only as a server during federated learning and only responsible for global weight aggregation and communication.

To reduce computational cost and obtain a compact representation suitable for MCUs, Mel Frequency Energy (MFE) features were extracted from the raw int16 arrays to decrease each 8000-point raw data to a 75-dimension feature vector. MFE was chosen over more complex spectral features to balance representational capacity and on-device efficiency. It is worth noting that here, to avoid inconsistencies introduced by numerical and implementation differences, the same MFE extraction pipeline was implemented both offline on the PC and on-device using C++, ensuring feature-space consistency between training and real-time inference. In addition, z-score normalization was applied to each feature dimension to reduce the dominance of absolute feature magnitudes, encouraging the model to focus on relative spectral patterns rather than scale differences. During this process, on the PC, both the extracted features and the corresponding training-set mean and standard deviation were stored and ready to be loaded on the MCUs. The latter were recused during real-time inference on the devices to minimize feature-space shift between real-time input and training samples.

A lightweight multilayer perceptron (MLP) model with two hidden layers was applied here, resulting in a total of 2,994 parameters. This model can potentially achieve a good expressive capacity while remain affordable for resource-constraint devices at the same time. The final MLP architecture consists of an input layer with 75 dimensions, followed by two hidden layers with 32 and 16 neurons using ReLU activation, and a 2-neuron output layer with softmax activation. Instead of explicitly modeling an unknown class, a confidence-based rejection strategy was applied based on the model's output probabilities. In this way, predictions with a max softmax probability below 0.6 were rejected to reduce false activations.

To support collaborative learning while preserving user's data privacy, a federated learning framework was implemented, with both MCUs initialized using identical random model weights. To further improve training stability and reduce the impact of high stochastic gradient noise caused by sample-wise weight updates, local training was performed for additional multiple epochs before federated aggregation. Specifically, each device first trained the model locally for 6 epochs using its own private voice data. After this warm-up phase, federated aggregation was triggered immediately and subsequently perform every 3 local epochs, with the aggregated global model used to initialize the next epoch of local training. These local weights were transmitted to a federated learning server via BLE. Afterward, the server aggregated the received local weights and generated updated global weights, which were then distributed back to all the devices.

Federated Proximal Optimization was adopted as the aggregation strategy to better handle non-IID user voice data and the limited size of local datasets. Unlike standard federated averaging, FedProx introduced a proximal constraint that restricted local model updates from drifting too far away from the current global model. By controlling how strongly the global model moved toward the weighted average of local models, FedProx reduced instability caused by different user accents and local optimization directions, leading to more stable training and improved generalization performance across users.

When it comes to the real-time inference, the inference mode is triggered only when a high - energy audio event is detected by the MCU's microphone.

III. Task 3: Evaluation and Results (Week 50, Contributor: Siwei Jiang & Junzhang Chen)

The system was evaluated under different testing scenarios to access classification behavior in both personalized and cross-speaker settings.

Before federated learning, each device produced correct responses primarily when used by its corresponding user, while cross-speaker usage resulted in a high rate of incorrect responses and near-chance performance, indicating strong speaker dependence. After applying federated learning, both users were able to operate either device with consistently correct responses, demonstrating improved cross-speaker generalization while maintaining on-device inference performance.

Deviations from the Initial Plan

The main deviation was introduced in the refinement of target classes. Initially, it was aimed to implement a 3-class classification model that explicit learns the 'unknown' class containing various irrelevant words. However, due to the relatively simple model structure and limited capacity, the unknown class could not be reliably separated from the off class during training. Moreover, its highly diverse nature introduced ambiguity in the feature space, which degraded the classification accuracy of the 'on' and 'off' classes. As a result, we switched to confidence-based rejection strategy, which proved more suitable for small on-device models. In addition, some simple time-domain features such as RMS energy and ZCR were considered for the feature extraction because of low computational cost. However, these features were found to be insufficiently robust under varying acoustic conditions. Therefore, normalized MFE was applied to capture deeper frequency spectral characteristics and provide more discriminative representations.

Links to the Final Outcome

- Video Demo: Please see the submission page.
- Git Repository: https://github.com/qingtongyou/ML-for-IoT-Final-Project